Object-Oriented Programming (OOP)

Java Backend Interview — Short Notes

1. The Four Pillars of OOP

1.1 Encapsulation

- Bundling data (fields) and methods that operate on that data into a single unit (class), and **restricting direct access** to internal state
- Achieved via **access modifiers**: private fields + public getters/setters
- Protects object integrity — validation logic lives inside the class, not scattered in caller code
- **Information hiding** — caller knows WHAT the object does, not HOW it does it

Java Example

```
public class BankAccount {
private double balance; // hidden
public void deposit(double amount) {
if (amount > 0) balance += amount; // validation inside
}
public double getBalance() { return balance; }
}
```

✔ Key: private fields, public methods, validation inside the class

1.2 Inheritance

- A class (child/subclass) acquires properties and behaviour of another class (parent/superclass)
- Promotes **code reuse** — common logic in parent, specialised logic in child
- Java uses **extends** (class) and **implements** (interface)
- Java supports **single inheritance** for classes; multiple inheritance via interfaces
- **IS-A relationship**: Dog IS-A Animal. Use inheritance only when IS-A is genuinely true
- **super** keyword — call parent constructor or overridden method
- **Method overriding** — child redefines parent method with same signature (@Override)

Java Example

```
class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal {
@Override
void speak() { System.out.println("Woof"); }
}
```

✔ Key: extends for classes, IS-A test, prefer composition over inheritance when in doubt

1.3 Polymorphism

- One interface, many implementations — same method call behaves differently depending on the object

Compile-time Polymorphism — Method Overloading

- Same method name, different parameter list in the **same class**
- Resolved at **compile time** by the compiler based on argument types

```
int add(int a, int b)
double add(double a, double b) // overloaded – different params
```

Runtime Polymorphism — Method Overriding

- Child class provides specific implementation of parent's method
- Resolved at **runtime** via dynamic dispatch (JVM decides which method to call)

```
Animal a = new Dog(); // reference type = Animal, object type = Dog
a.speak(); // calls Dog.speak() at runtime – dynamic dispatch
```

Polymorphism via Interfaces

```
interface Shape { double area(); }
class Circle implements Shape { public double area() { return Math.PI*r*r; } }
class Square implements Shape { public double area() { return side*side; } }
Shape s = new Circle(5); // same type, different behaviour
```

✔ Key: overloading = compile time, overriding = runtime. Runtime polymorphism is the more powerful and commonly tested concept

1.4 Abstraction

- Hiding **implementation details**, exposing only essential features — what an object does, not how
- Reduces complexity — caller doesn't need to know internal workings
- Achieved via **abstract classes** and **interfaces**

Abstract Class

- Can have abstract methods (no body) AND concrete methods (with body)
- Cannot be instantiated directly
- Use when: subclasses share common state/behaviour + some methods must be overridden

```
abstract class Vehicle {
abstract void start(); // subclass must implement
void fuel() { /* common */ } // shared implementation
}
```

Interface

- Defines a contract — all methods abstract by default (Java 8+: can have default/static methods)
- A class can implement multiple interfaces — Java's answer to multiple inheritance
- Use when: defining a capability/contract across unrelated classes

```
interface Flyable { void fly(); }
interface Swimmable { void swim(); }
class Duck implements Flyable, Swimmable { ... } // multiple interfaces
```

✔ Key: abstraction hides HOW. Interface = pure contract. Abstract class = partial implementation

2. Interface vs Abstract Class

Feature	Interface	Abstract Class
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Abstract by default; default & static allowed (Java 8+)	Mix of abstract and concrete methods
Fields	public static final only (constants)	Any access modifier, instance fields allowed
Constructors	No constructors	Has constructors (called via super)
Multiple inheritance	A class can implement many interfaces	A class can extend only one abstract class

Access modifiers	Methods implicitly public	Any access modifier on methods
State	No state (no instance variables)	Can have state (instance variables)
Use when	Defining a contract/capability across unrelated classes	Sharing code among closely related classes
Example	Serializable, Comparable, Runnable	HttpServlet, AbstractList

✔ Rule of thumb: prefer interfaces for abstraction. Use abstract class only when you need to share implementation/state

3. Access Modifiers

Modifier	Same Class	Same Package	Subclass	Everywhere
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

- **private** — encapsulation; most restrictive; use for fields
- **default (package-private)** — no keyword; visible within same package only
- **protected** — visible in subclasses even in different packages; use for inheritable members
- **public** — visible everywhere; use for API methods

4. Key OOP Concepts in Java

4.1 Constructor

- Special method called when object is created — initialises the object state
- **Default constructor** — compiler generates if none defined; no args, empty body
- **Constructor overloading** — multiple constructors with different parameters
- **Constructor chaining** — this() calls another constructor in same class; super() calls parent
- Constructors are NOT inherited and NOT polymorphic

```
class Car { Car() {} Car(String model) { this.model = model; } }
```

4.2 this & super Keywords

- **this** — refers to current object instance; disambiguates field vs parameter; calls another constructor
- **super** — refers to parent class; access parent fields/methods; call parent constructor (must be first statement)

```
class Dog extends Animal {
Dog(String name) { super(name); } // parent constructor
void speak() { super.speak(); System.out.println("Woof"); }
}
```

4.3 static Keyword

- **static field** — shared across all instances of the class (class-level, not object-level)
- **static method** — belongs to class, not instance; cannot access non-static members; cannot be overridden
- **static block** — runs once when class is loaded; used for static initialisation
- **static class** — nested static class; can be instantiated without outer class instance

```
class Counter { static int count = 0; // shared across all Counter instances
Counter() { count++; } }
```

4.4 final Keyword

• **final variable** — constant; must be initialised once; cannot be reassigned

• **final method** — cannot be overridden by subclasses

• **final class** — cannot be extended (e.g., String, Integer, all wrapper classes)

```
final class ImmutablePoint { final int x, y; ... }
```

4.5 Method Overloading vs Overriding

	Overloading	Overriding
Where	Same class	Parent + Child class
Signature	Different parameter list	Same signature
Return type	Can differ	Same (or covariant)
Resolution	Compile time	Runtime (dynamic dispatch)
Annotation	None needed	@Override (recommended)
static/final	Can overload static/final	Cannot override static/final/private

4.6 Object Class Methods

• Every class implicitly extends **java.lang.Object**

• **equals()** — logical equality; override when you define value-based equality (default: reference equality ==)

• **hashCode()** — MUST override together with equals(). Equal objects MUST have same hashCode

• **toString()** — string representation; override for meaningful output

• **clone()** — shallow copy; implement Cloneable; prefer copy constructor

• **getClass()** — returns runtime class; **instanceof** checks type

```
@Override public boolean equals(Object o) {
if (this == o) return true;
if (!(o instanceof Person)) return false;
return this.id == ((Person)o).id; }
```

4.7 Composition vs Inheritance

• **Inheritance** — IS-A: Dog IS-A Animal. Tightly coupled; changes in parent break child

• **Composition** — HAS-A: Car HAS-A Engine. Loosely coupled; more flexible; preferred

• *"Favour composition over inheritance"* — GoF Design Patterns

```
// Composition (preferred)
class Car { private Engine engine; // HAS-A
Car() { this.engine = new Engine(); } }
```

4.8 Immutability

• Object whose state cannot change after creation — inherently thread-safe

• Make class **final**, all fields **private final**, no setters, deep copy in constructor and getters

• Examples: String, Integer, LocalDate — all Java wrapper/value classes are immutable

```
public final class Money {
private final double amount;
private final String currency;
public Money(double amount, String currency) { this.amount=amount; this.currency=currency; }
// no setters — immutable
```

```
}
```

4.9 Coupling & Cohesion

- **Coupling** — degree of dependency between classes. **Low coupling** = good; changes in one class don't ripple
- **Cohesion** — degree to which a class has a single, well-focused purpose. **High cohesion** = good
- Goal: **Low coupling + High cohesion** = maintainable, testable design

4.10 Java 8+ OOP Features

- **Default methods in interfaces** — provide method implementation in interface; enables backward compatibility
- **Static methods in interfaces** — utility methods on interface; not inherited
- **Functional interfaces** — interface with exactly one abstract method; usable as lambda target
- **Lambda expressions** — anonymous implementation of functional interface
- **Records (Java 16)** — immutable data class; auto-generates constructor, getters, equals, hashCode, toString
- **Sealed classes (Java 17)** — restrict which classes can extend/implement a class/interface

```
// Record — replaces verbose immutable class
record Point(int x, int y) {} // auto: constructor, getters, equals, hashCode, toString
// Sealed class
sealed class Shape permits Circle, Square, Triangle {}
```

5. SOLID Principles

S — Single Responsibility Principle (SRP)

- A class should have only ONE reason to change — one job, one responsibility
- **Example:** UserService handles business logic. UserRepository handles DB. UserController handles HTTP. Each has one job
- **Violation:** Violation: A class that handles business logic, DB saving, AND email sending

O — Open/Closed Principle (OCP)

- Open for extension, closed for modification — add new behaviour without changing existing code
- **Example:** Add a new PaymentMethod (CryptoPayment) by implementing PaymentMethod interface — no change to PaymentProcessor
- **Violation:** Violation: Adding if-else blocks to existing method every time new payment type is added

L — Liskov Substitution Principle (LSP)

- Subclass must be substitutable for its parent class without breaking the program
- **Example:** If method accepts Animal, it must work correctly when passed a Dog or Cat — no surprises
- **Violation:** Violation: Square extends Rectangle but overriding setWidth breaks the Rectangle contract (area = w*h fails)

I — Interface Segregation Principle (ISP)

- No class should be forced to implement methods it doesn't use — split fat interfaces into specific ones
- **Example:** Separate Flyable, Swimmable, Runnable interfaces instead of one Animal interface with all methods
- **Violation:** Violation: Implementing a 20-method interface when class only needs 2 methods

D — Dependency Inversion Principle (DIP)

- High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces)
- **Example:** OrderService depends on PaymentGateway interface, not on StripePayment directly — easy to swap
- **Violation:** Violation: OrderService directly instantiates StripePayment — hard to test, hard to change

6. Key Design Patterns (OOP Context)

Creational Patterns — Object Creation

Pattern	Intent	Java Example
Singleton	One instance throughout app lifetime	Spring beans (default), Logger, Config
Factory	Delegate object creation to subclass	ShapeFactory.create('circle') — returns Shape
Builder	Build complex object step by step	StringBuilder, Lombok @Builder, HttpRequest.newBuilder()
Prototype	Clone existing object	Object.clone(), copy constructors

Structural Patterns — Object Composition

Pattern	Intent	Java Example
Adapter	Convert interface of one class to another expected by client	Arrays.asList() wraps array as List
Decorator	Add behaviour without changing class — wraps object	BufferedReader wraps FileReader; Spring AOP
Proxy	Control access to another object	Spring AOP proxies, lazy loading in Hibernate
Facade	Simplified interface to complex subsystem	SLF4J facade over Log4j/Logback; JPA over JDBC
Composite	Treat individual objects and groups uniformly	File system: File and Directory both implement Component

Behavioural Patterns — Object Communication

Pattern	Intent	Java Example
Strategy	Define family of algorithms, make them interchangeable	Comparator — different sort strategies
Observer	Notify multiple objects on state change (pub/sub)	EventListener in Spring, Java PropertyChangeListener
Template	Define skeleton of algorithm; subclasses fill steps	AbstractList, JdbcTemplate, Spring's doGet/doPost
Command	Encapsulate request as object — undo/redo support	Runnable, Spring @Async
Chain of Resp	Pass request through chain of handlers until one handles it	Spring Security filter chain, Servlet filters
Iterator	Sequentially access elements without exposing internals	java.util.Iterator, enhanced for-loop

7. Frequently Asked & Tricky Interview Questions

Four Pillars

Q1. What are the 4 pillars of OOP?

→ Encapsulation (data hiding), Abstraction (hiding implementation), Inheritance (IS-A reuse), Polymorphism (one interface, many behaviours)

Q2. Difference between Abstraction and Encapsulation?

→ Abstraction = hides complexity, shows only essential features (WHAT). Encapsulation = hides internal data, bundles data+methods (HOW is protected). Both use access modifiers but serve different purposes

Q3. What is polymorphism? Types?

→ One interface, many implementations. Compile-time (overloading — resolved by compiler). Runtime (overriding — resolved by JVM via dynamic dispatch)

Q4. Can we achieve polymorphism without inheritance?

→ Yes — via interfaces. Unrelated classes implement the same interface and behave polymorphically

Q5. What is dynamic method dispatch?

→ JVM determines at runtime which overridden method to call based on the actual object type, not the reference type

Inheritance

Q6. Why does Java not support multiple inheritance for classes?

→ Avoids the Diamond Problem — ambiguity when two parents have the same method. Java allows multiple interface implementation instead

Q7. What is the Diamond Problem?

→ If class D extends B and C, and both B and C override method from A, which version does D inherit? Java solves this by prohibiting multiple class inheritance

Q8. Can a constructor be inherited?

→ No. Constructors are not inherited. But parent constructor is called implicitly (super()) or explicitly

Q9. What happens if you don't call super() in child constructor?

→ Java automatically inserts super() as first statement to call parent's no-arg constructor. If parent has no no-arg constructor, compilation fails

Q10. Can static methods be overridden?

→ No — static methods are resolved at compile time, not runtime. Child can redeclare (hide) but not override. @Override on static method = compile error

Q11. What is method hiding?

→ When child declares a static method with same signature as parent's static method. Not true overriding — resolved at compile time based on reference type

Q12. Composition vs Inheritance — when to use which?

→ Inheritance: genuine IS-A with intent to override behaviour. Composition: HAS-A or CAN-DO. Prefer composition — less coupling, more flexible

Interface & Abstract Class

Q13. Interface vs Abstract Class — key differences?

→ Interface: no state, multiple implementation, all methods public. Abstract class: has state, single inheritance, any access modifier, mix of abstract/concrete methods

Q14. When would you use abstract class over interface?

→ When related classes share common state (fields) or partial implementation. e.g., Template Method pattern — skeleton in abstract class, steps in subclasses

Q15. Can interface have constructors?

→ No. Interfaces cannot have constructors. They can have default and static methods (Java 8+) and private methods (Java 9+)

Q16. What is a default method in interface? Why added?

→ Method with body in interface (default keyword). Added in Java 8 to allow adding methods to interfaces without breaking all existing implementations

Q17. Can interface extend another interface?

→ Yes — using extends. Interface can extend multiple interfaces. A class implementing child interface must implement all methods from the chain

Q18. What is a marker interface?

→ Empty interface with no methods — used to tag/mark a class. e.g., Serializable, Cloneable, RandomAccess. Modern alternative: annotations

Q19. What is a functional interface?

→ Interface with exactly one abstract method. Annotated @FunctionalInterface. Can be used as lambda target. e.g., Runnable, Comparator, Predicate

Keywords & Modifiers

Q20. What is the difference between == and equals()?

→ == compares references (memory addresses). equals() compares logical value. For String, always use equals(). Override equals() for custom value equality

Q21. Why must you override hashCode() when you override equals()?

→ Contract: equal objects must have same hashCode. If not, objects placed in HashMap/HashSet won't be found — they'll land in wrong bucket

Q22. What does final mean for class, method, variable?

→ Class: cannot be extended. Method: cannot be overridden. Variable: cannot be reassigned (but object state can change if mutable)

Q23. Can a final class have abstract methods?

→ No — contradictory. abstract requires subclassing; final prevents it

Q24. What is the difference between static and instance methods?

→ Static: belongs to class, no 'this', cannot access instance members, not polymorphic. Instance: belongs to object, has 'this', polymorphic

Q25. What is the purpose of the this keyword?

→ Refers to current object. Disambiguates field vs parameter (this.name = name). Calls another constructor in same class (this()). Passes current object as argument

Q26. What is the difference between pass-by-value and pass-by-reference in Java?

→ Java is always pass-by-value. For primitives: value is copied. For objects: reference (address) is copied — caller's original reference unchanged but object state can be modified

Advanced OOP

Q27. What is immutability? How to make a class immutable?

→ State cannot change after creation. Make class final, fields private final, no setters, deep copy mutable objects in constructor/getters

Q28. Why is String immutable in Java?

→ Security (can't modify string passed to network/file), thread safety (no synchronisation needed), String pool (can share instances), caching hashCode

Q29. What is the difference between shallow copy and deep copy?

→ Shallow: copies object but shares references to nested objects (original and copy share internals). Deep: recursively copies all nested objects — fully independent

Q30. What is coupling and cohesion?

→ Coupling: dependency between classes — low coupling = good. Cohesion: focus of a class — high cohesion = single responsibility = good. Goal: low coupling, high cohesion

Q31. What is the Object class in Java?

→ Root of all Java class hierarchy. Every class implicitly extends Object. Provides: equals(), hashCode(), toString(), clone(), getClass(), wait(), notify(), finalize()

Q32. What is method hiding vs method overriding?

→ Overriding: instance methods, runtime polymorphism, actual object type determines call. Hiding: static methods, compile-time, reference type determines call

Q33. Can abstract class have a constructor?

→ Yes — called by subclass constructor via super(). Used to initialise common state. Cannot be called directly with new

Q34. What is covariant return type?

→ Overriding method can return a subtype of parent method's return type. e.g., parent returns Animal, child can return Dog. Allowed since Java 5

SOLID & Design Patterns

Q35. Explain the Single Responsibility Principle with example

→ One class, one reason to change. UserService = business logic only. Violation: UserService also sends emails and saves to DB

Q36. What is the Open/Closed Principle?

→ Open for extension (add new code), closed for modification (don't change existing). Use interfaces/abstract classes + polymorphism to achieve this

Q37. Explain Liskov Substitution Principle. Classic violation?

→ Subclass must be substitutable for parent without breaking behaviour. Classic violation: Square extends Rectangle — setting width also sets height, breaking Rectangle contract

Q38. What is Interface Segregation Principle?

→ Prefer many small specific interfaces over one large fat interface. No class should implement methods it doesn't need

Q39. Dependency Inversion — how is it different from Dependency Injection?

→ DIP is a principle (depend on abstractions). DI is a pattern/technique that implements DIP (inject dependencies from outside). Spring IoC implements both

Q40. What is the Singleton pattern? Thread safety concern?

→ One instance per JVM. Thread safety issue if using lazy init without synchronisation. Solutions: enum singleton, double-checked locking, or eager init with static field

Q41. Difference between Factory and Abstract Factory pattern?

→ Factory: creates one type of object via subclass. Abstract Factory: creates families of related objects without specifying concrete classes

Q42. What is the Strategy pattern?

→ Define algorithms as separate classes implementing same interface. Switch algorithm at runtime. e.g., Comparator for sorting — swap strategy without changing sorting code

Q43. What is the Observer pattern?

→ Subject maintains list of observers; notifies all on state change. Same as pub/sub. Java: EventListener, PropertyChangeListener. Spring: ApplicationEvent

Q44. Template Method vs Strategy pattern?

→ Template: defines skeleton in abstract class, subclasses fill steps — inheritance-based. Strategy: delegates algorithm to injected strategy object — composition-based

Q45. What is the Decorator pattern? How is it different from Inheritance?

→ Decorator wraps object to add behaviour at runtime without subclassing. More flexible than inheritance — can stack multiple decorators. e.g., Java I/O streams

Java OOP Specifics

Q46. What is the difference between an inner class and a static nested class?

→ Inner class: non-static, has implicit reference to outer class instance, can access outer instance members. Static nested: no outer reference, instantiated independently

Q47. What are Java Records (Java 16)?

→ Concise immutable data class. Auto-generates constructor, getters, equals, hashCode, toString. Replaces verbose DTO/value objects. Cannot extend other classes

Q48. What are Sealed Classes (Java 17)?

→ Restrict which classes can extend/implement a sealed class/interface using permits clause. Enables exhaustive pattern matching. e.g., sealed class Shape permits Circle, Square

Q49. What is the difference between Comparable and Comparator?

→ Comparable: class defines its natural ordering (compareTo in same class). Comparator: external ordering logic (compare in separate class). Use Comparator for multiple sort orders

Q50. What happens when equals() is overridden but hashCode() is not?

→ Two logically equal objects may have different hashCodes. They'll be placed in different HashMap buckets — get() after put() won't find the object. Critical bug in collections